第2章 线性表

2.1线性表的定义

2.2线性表的顺序存储结构

2.3线性表的链式存储结构

2.4一元多项式的表示与相加

2.1 线性表的定义

2.1.1 线性表的概述

2.1.2 线性表的抽象数据类型

2.1.3 线性表的相关操作

2.1.1 线性表的概述

➢线性表的逻辑定义

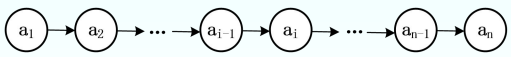
•n(n≥0)个属于同一数据类型的数据元素$a_1, a_2, \ldots\ldots, a_n$组成的有限序列，称为线性表。

L=(D,R)

其中，D={$a_1, a_2, \ldots\ldots, a_n$}

R={< a_{i-1}, a_i>| $a_{i-1}, a_i \in D, 2 \leqslant i \leqslant n$ }

•当n=0时，为空表。



➢ 线性表的特点

•有且仅有一个“头”元素

•有且仅有一个“尾”元素

•其它元素有且仅有一个前驱和一个后继元素

2.1.2 线性表的抽象数据类型

ADT List{

数据对象：任意数据元素的集合

D={ a_i| $a_i \in$ ElemSet, i=1,2,…,n, n≥0}

数据关系：除第一个和最后一个外，每个元素有唯一的直接前驱和唯一的直接后继。

R={<a_{i-1}, a_i>| $a_{i-1}, a_i \in D$, i=1,2,…,n, n≥0}

基本操作：

ListInit(L) //构造一个空的线性表L

ListLength(L) //求线性表长度

ListGet(L,i) //读取第i个元素

线性表的抽象数据类型（续）

ListLocate(L,x) //查找值为x的元

ListPrior(L,e) //求元素e的前驱

ListNext(L, e) //求元素e的后继

ListInsert(L,i, e) //在第i个元素前插入元素e

ListDelete(L,i) //删除第i个元素

ListEmpty(L) //判断表是否空

ListClear(L) //清空表中元素

}ADT List

2.1.3 线性表的相关操作

◆可调用ADT中的“基本操作”实现更复杂的操作

1.线性表的遍历

➢遍历——访问某种数据结构中的每一个元素，并且每个元素只访问一次。

➢线性表的遍历:

①空表？→不遍历

②非空表？→表长→访问每个元素

线性表的遍历算法

```
ListTraverse（L）                          //遍历线性表中的元素
{
   if（ListEmpty（L））                    //判断表是否为空
      printf（“这是一个空表”）；
   else
      for（i=1；i<=ListLength（L）；i++）
         visit（ListGet（L，i））；          //访问线性表中的元素i
}
```

2. 线性表的合并

- 合并：将两个线性表La和Lb合并成一个长度为ListLengh(La)+ ListLength(Lb)的线性表Lc。

1）直接合并：将线性表Lb中的元素插入到线性表La中。

2）保序合并：将两个有序的表La和Lb归并为一个新的线性表Lc，并且保证线性表Lc仍然是有序的。

参见P20　例2-1　例2-2

2.2 线性表的顺序存储结构及其操作

2.2.1 线性表的顺序存储结构

2.2.2 相关操作的实现

2.2.3 顺序存储结构的分析

2.2.1 线性表的顺序存储结构

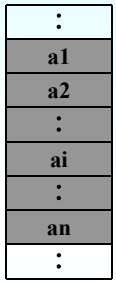
- 线性表的顺序存储

——用一组地址连续的存储单元依次存储线性表中的数据元素。

- 设每一元素占L个单元，且已知第一个元素的地址，有

$Loc(ai)=Loc(a_1)+(i-1)*L$

存储状态



2.2.2 顺序表的相关操作

1. 顺序表的初始化（定义空表）
2. 顺序表的求长度
3. 顺序表的取元素（查找第i个元素）
4. 顺序表的定位（按值查找）
5. 顺序表的求前驱（先定位某个元素，该元素的前驱即所求）
6. 顺序表的求后继（先定位某个元素，该元素的后继即所求）
7. 顺序表的前插
8. 顺序表的删除
9. 顺序表的判空（表长是否为0）
10. 顺序表的置空（即初始化）
11. 顺序表的遍历
12. 顺序表的合并

顺序表的动态分配（书P22）

由于线性表的长度可变，所需最大存储空间随问题不同而不同，因此，在C语言中可用动态分配的数组来描述线性表。

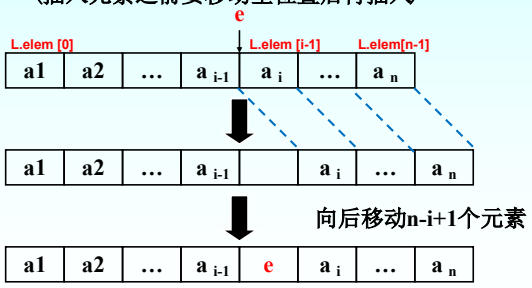
```
#define  LIST_INIT_SIZE   100  //线性表存储空间的初始分配量
#define  LISTINCREMENT 10   //线性表存储空间的分配增量
typedef  struct {
        ElemType  *elem;    //存储空间的基地址
        int   length ;        //当前长度（即数据元素的个数）
        int   listsize;       //当前分配的存储容量（以sizeof（ElemType）为单位）
 } SqList ;
```

注意：　ElemType——代表所有可能的数据类型。

1. 顺序表的插入

在线性表中第i个元素之前插入一个元素e。

（插入元素之前要移动空位置后再插入）



注意：C语言中数组的下标从“0”开始，因此表中的第i个数据是L.elem[i-1]！

顺序表插入的算法（P24 算法2.4）

```
Status Insert_Sq (Sqlist &L，int i，ElemType e)  //1≤i≤L.length+1合法
{
    if ( i<1||i>L.length+1)  return  ERROR;  //插入位置非法
    if (L.length>=L.listsize)
                                        //当前存储空间已满，增加分配
{
    newbase=(ElemTpye *)realloc(L.elem,
                (L.listsize+LISTINCREMENT)* sizeof(ElemType));
    if(!newbase) exit(OVERFLOW);      //存储分配失败
    L.elem=newbase;  //新基址
    L.listsize+=LISTINCREMENT;  //增加存储容量
}
```

顺序表插入的算法（续）

```
    q=&(L.elem[i-1]);    //q是插入位置
    for  ( p=&(L.elem[L.length-1]); p>=q; --p)
        *(p+1)=*p;        //插入位置以后的元素后移
    *q=e;                 // 插入e
    ++L.length;           // 表长增1
    return  OK ;
}
```

注意：
（1）C语言中，q、*q和&q的区别？
（2）函数realloc()的含义？
（3）++p和p++的区别？

顺序表插入的算法评价

设pi是在第i个元素前插入一个元素的概率，则在表长为n的表中插入一个元素所需移动次数的平均值为：

$$E_{is} = \sum_{i=1}^{n+1} p_i (n - i + 1)$$

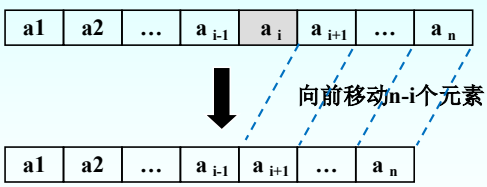
假定在顺序表中任一位置上插入一个元素的概率相等，且Σpi＝1，有pi=1/(n+1)，则

$$E_{is} = \frac{1}{n+1} \sum_{i=1}^{n+1} (n - i + 1) = \frac{n}{2}$$

结论：在顺序表中插入一个元素时，平均需移动表的一半元素，其时间复杂度是O(n)。

2. 顺序表的删除

删除长度为n的线性表中第i个元素。



顺序表删除的算法（P24-25算法2.5）

```
Status ListDelete_Sq(Sqlist &L，int i, ElemType &e) // 1≤i≤L.length合法
{
    if ( i<1||i>L.length ) return ERROR ;   // 被删除位置非法
    p=&(L.elem[i-1]);     //p为被删除元素的位置
    e=*p;                 // 保存被删除元素的值
    q=L.elem+L.length-1;  //q是表尾元素的位置
    for ( ++p;  p<=q; ++p)
        *(p-1)=*p;        //被删除元素之后的元素前移
    --L.length;     //表长减1
    return OK;
}
```

顺序表删除的算法评价

设qi是删除第i个元素的概率，则在表长为n的表中删除一个元素所需移动次数的平均值为：

$$E_{de} = \sum_{i=1}^{n} q_i (n - i)$$

假定在顺序表中任一位置上删除一个元素的概率相等，且Σqi＝1，有qi=1/n，则

$$E_{de} = \frac{1}{n}\sum_{i=1}^{n} (n - i) = \frac{n-1}{2}$$

结论：在顺序表中删除一个元素时，平均约需移动表的一半元素，其时间复杂度是O（n）。

2.2.3 顺序存储结构的分析

(1)实现方法简单。

(2)每个元素的存储位置可以用一个简单的公式来计算。

(3)遍历操作和定位操作都以线性的时间执行，取元素操作只需要花费常数的时间。

(4)顺序存储的空间是静态分配的，需要事先指定MAXSIZE的大小，因此，在事先不知道线性表大小的情况下，很容易造成空间的浪费。

(5)插入与删除操作的花费是很昂贵的。

2.3 线性表的链式存储结构及其操作

2.3.1 线性链表与相关操作实现

2.3.2 双向链表与相关操作实现

2.3.3 循环链表与相关操作实现

2.3.4 链式存储结构的分析

2.3.1 线性链表与相关操作实现

➢线性表的链式存储

——用连续（或不连续）的存储单元存储线性表中的数据元素，但元素间的逻辑关系需要依靠“指针”指明。

➢设指针p指向某一结点（称该结点为结点p），



其中

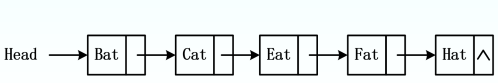
p ->data —— 存放p所指向结点的数据域（存放该结点本身的数据）

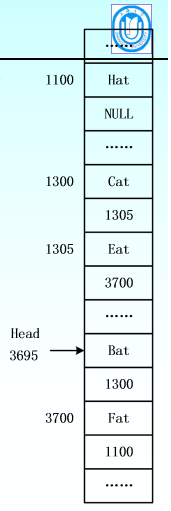
p ->next —— 存放p所指向结点的指针域（存放该结点的下一结点地址）

头指针和链表

□链表是由表头唯一确定，因此单链表可以用头指针的名字来命名。

例：某链表的物理存储方式如右图所示，其对应的逻辑状态是什么？





结点的描述

C语言中用带指针的结构体类型来描述

```
typedef  struct  Lnode
   {  ElemType  data;    /*数据域，保存结点的值*/
      struct  LNode *next;   /*指针域*/
   }LNode;        /*结点的类型 */
```

结点的实现

（1）结点是通过动态分配和释放来的实现，即需要时分配，不需要时释放。

（2）实现分配和释放时，分别使用C语言提供的标准函数：

malloc()，sizeof()，free()。

动态分配　　p=(LNode*)malloc(sizeof(LNode));

函数malloc分配了一个类型为LNode的结点变量的空间，并将其首地址放入指针变量p中。

动态释放　　free(p)；

系统回收由指针变量p所指向的内存区。P必须是最近一次调用malloc函数时的返回值。

最常见的基本操作

（1）结点的赋值

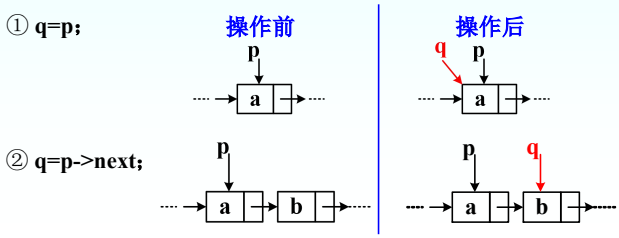
LNode *p;

p=(LNode*)malloc(sizeof(LNode));

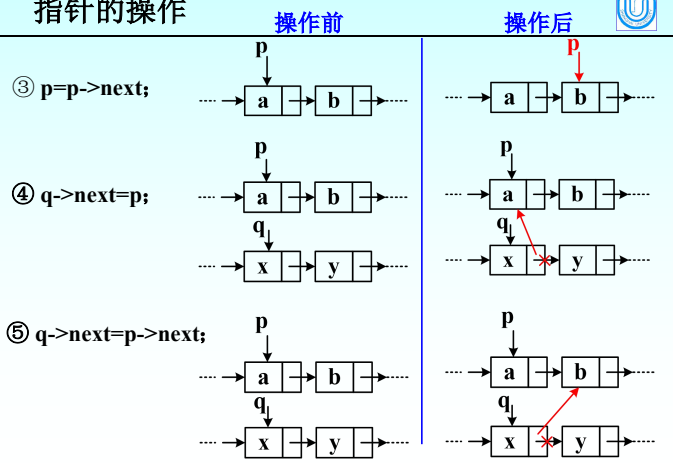
p->data=20; p->next=NULL;



（2）指针的操作



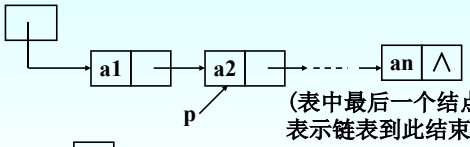
指针的操作



单链表

➢单链表——结点包含一个指针，指明直接后继元素。

head(头指针，存放链表的第一个结点的地址)



(表中最后一个结点指针为空，表示链表到此结束)

head ∧（空表）

特点：只能在一个方向上遍历其后的元素。

例：在上图的单链表中，指针p指向值为a2的结点。

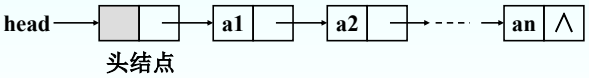
head->data=_____; head->next=_____;

head->next->data=_____。

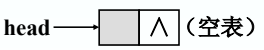
具有头结点的单链表

➢ 在链表的第一个结点之前附加一个与其它结点完全相同的结点，称为*头结点*。

头结点{ 数据域——空的，或用来存放其它信息，如链表的长度
指针域——指向链表的第一个结点

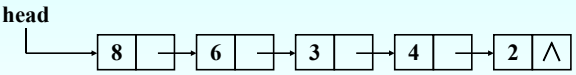


头结点

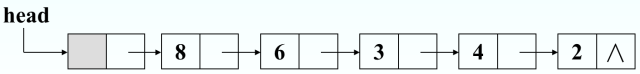


具有头结点的单链表（例题）

例：在如下带头结点的单链表中，



（1）head->next->next->data=_____。



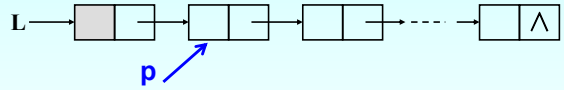
（2）head->next->next->data=_____。

单链表的相关操作

1. 单链表的初始化
2. 单链表的求表长
3. 单链表的取元素
4. 单链表的定位
5. 单链表的插入
6. 单链表的删除
7. 创建单链表
8. 单链表的保序合并

实质就是“插入”

单链表的求表长



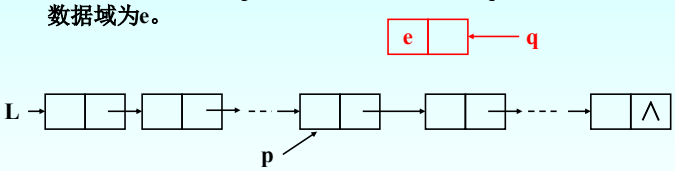

```
int LinkedListLength（LinkedList L）        //带头结点的单链表求表长
{   LinkedList *p;
    p=L ->next;
    int n=0;                                //n是表长
    while （p！=NULL）                      //判断是否到达表尾
    {    n++;
         p=p->next;    }                    //p指向下一个结点
    return n;
}
```

思考题：（1）如改成指令：p=L，上述算法如何改写？

（2）不带头结点的单链表的表长？

单链表的插入——后插

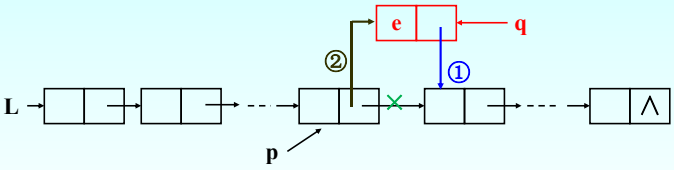
➢在单链表L的结点p之后插入一个新的结点q，该结点的数据域为e。



（1）生成一新结点q，存放值e

```
q=(LNode *)malloc(sizeof(LNode));
if(q=NULL)
{    printf(“申请空间失败！”);
     exit(0);    }
q->data=e;
```

单链表的后插



（2）在结点p之后插入结点q

① q->next=p-> next;

② p->next=q;

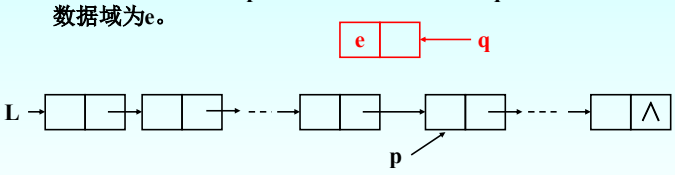
先确定新指针

再修改旧指针

思考题： ①和②两个步骤是否可以颠倒？ ？

单链表的插入——前插

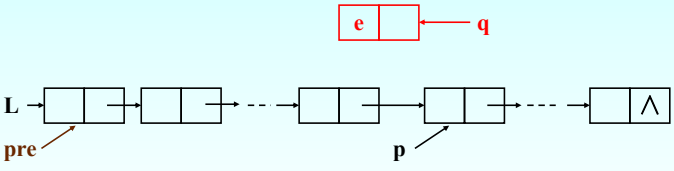
➢在单链表L的结点p之前插入一个新的结点q，该结点的数据域为e。



（1）生成一新结点q，存放值e

```
q=(LNode *)malloc(sizeof(LNode));
if(q=NULL)
{    printf(“申请空间失败！”);
     exit(0);    }
q->data=e;
```

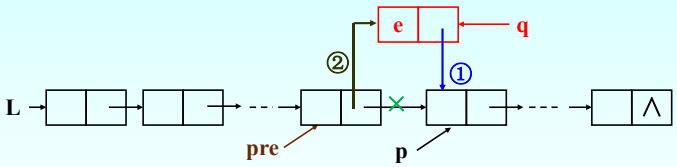
单链表的前插



（2）查找到结点p的前驱结点pre

```
pre=L;
while((pre!=NULL)&&(pre->next!=p))   //找到前驱结点pre或达到表尾，
                                     跳出该循环
     pre=pre->next;
```

单链表的前插

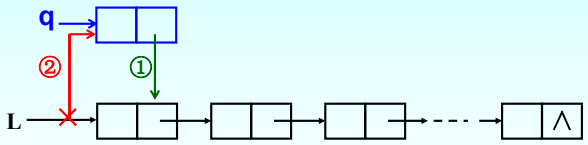


（3）插入结点q（即在结点pre之后插入）

① q->next=pre-> next;　或q->next=p;

② pre->next=q;

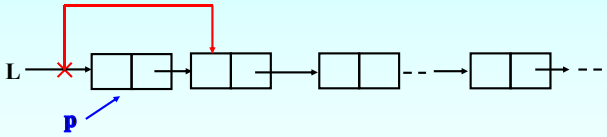
在单链表的表头插入新结点（建立新表）



① q->next=L;

② L=q;

单链表的删除——删除第一个结点



•p指向待删除的第一个结点

p=L;

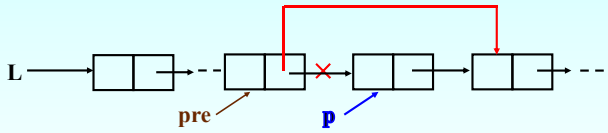
•删除第一个结点——修改头指针L

L=L->next;

•释放p结点

free（p）;

单链表的删除——删除表中其他任一结点p



•找到p的前驱结点pre

（省略）

•删除p结点——修改指针

pre->next=p->next;　或 pre->next=pre->next->next;

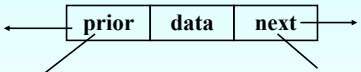
•释放p结点

free（p）;

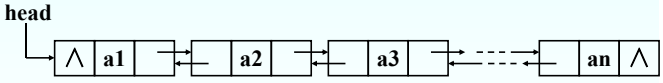
结论：在单链表中插入或删除一个结点时，仅需要改变一个或两个指针，而无需移动元素。

2.3.2 双向链表与相关操作实现

表中的结点有两个指针域，其中一个指向直接后继，另一个指向直接前趋。



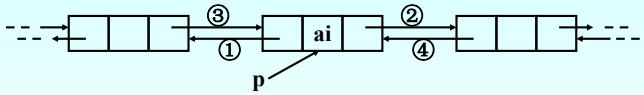
存放相邻前一结点的地址　　存放相邻下一结点的地址



head ∧（空表）

特点：可在两个方向上遍历其后或其前的元素。

双向链表中各指针之间的关系



① p-> prior

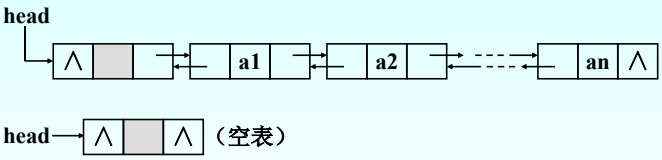
② p->next

③ p->prior->next

④ p->next->prior

p = p->prior->next = p->next->prior

具有头结点的双向链表

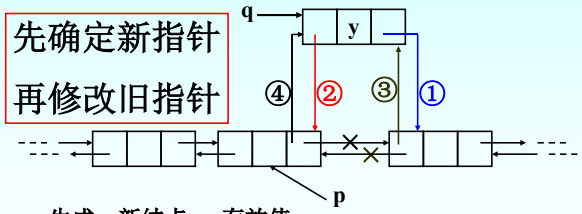


双向链表的特点：

以增加指针的存储空间为代价，提高双向链表的工作效率。从某一个结点出发，既可向后查找其后继结点，又可向前查找前趋结点。

双向链表的插入

例：将值为y的结点插在p结点后。



•生成一新结点q，存放值y

q=(LNode *)malloc(sizeof(LNode)); q->data=y;

•插入

①q-> next=p-> next;

② q->prior=p;

③ q->next-> prior=q;

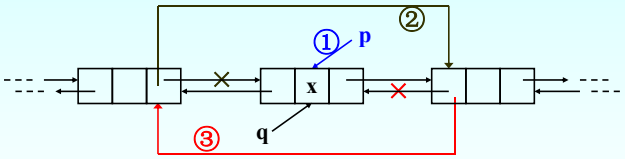
④ p->next=q;

思考题：

p结点之前插入呢？

双向链表的删除

例：删除q结点。



•删除

~~① p->next=q;~~

② q->prior->next=q-> next;

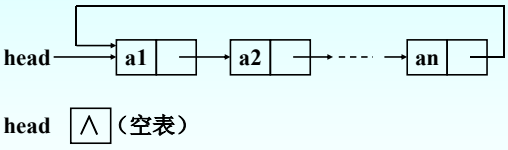
③ q->next->prior=q->prior;

•回收结点p

~~free (p);~~ free (q);

2.3.3 循环链表

使单链表的最后一个元素结点的指针不为空，而指向表的第一个结点，使链表构成环状。

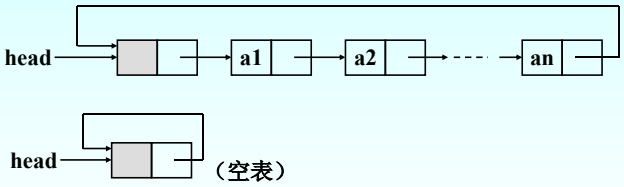


循环链表的特点：可在任何位置上遍历整个链表，显著提高查找的效率。

在单链表中，查找某个结点，总要从头开始

在循环链表中，可从表中任一结点出发查找

具有头结点的循环链表



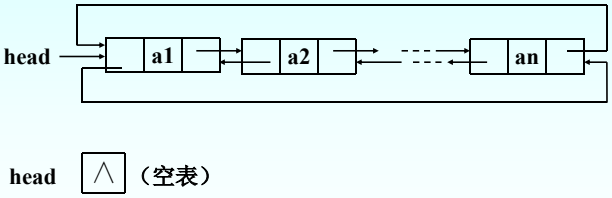
思考题：

如何判断单链表与循环链表的表尾？

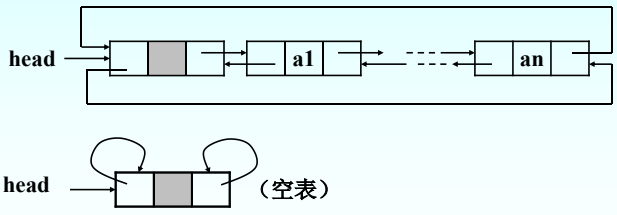
解：单链表中，判断某个结点的指针域是否为空；

循环链表中，判断某个结点的指针域是否等于头指针。

双向循环链表



具有头结点的双向循环链表



2.3.4 链式存储结构的分析

链式存储结构优、缺点

(1)链表的存储空间是动态分配的，只要内存尚有空间，就不会产生溢出。

(2)便于实现插入和删除操作。

(3)由于链表的不连续存储，因此它的内容分散，有时会导致调试的不方便。

(4)链表中的每个结点既有数据域又有指针域，增加了线性表的存储开销。

单链表、双向链表、循环链表的特点

(1)在单链表中查找结点时，需要从头指针开始遍历，增加了时间。

(2)双向链表解决了单链表的单向性问题，使得链表查找前驱结点变得简单，但是它增加了结点的存储空间，使维护操作变得困难。

(3)循环链表解决了单链表必须从头指针进行遍历的限制。

选择存储结构遵循的原则

(1)当线性表的长度变化较大，难以估计其存储的规模时，一般采用链表作为存储结构。

(2)当线性表的长度变化不大，易于事先确定其大小时，宜采用顺序表作为存储结构，这样可以节省存储空间。

(3)当线性表的主要操作是查找，并且很少进行插入和删除操作时，宜采用顺序表作为存储结构。

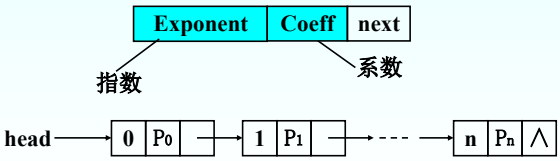
(4)若线性表需要频繁地进行插入和删除操作时，为了提高性能，宜采用链表作为存储结构。

2.4 一元多项式的表示与相加

一元多项式可表示为

$$P_n(x)=P_0+P_1*x+P_2*x^2+\ldots+P_i*x^i+\ldots+P_n*x^n$$

每一项用一结点表示: *两个数据域，一个指针域*



一元多项式的相加

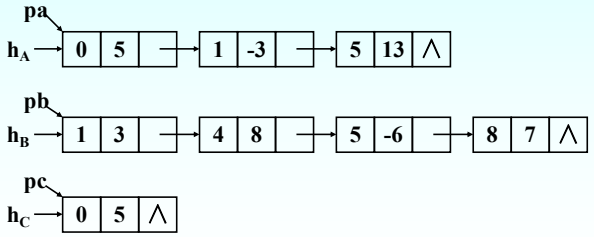
一元多项式加法规则：

- 两个多项式中指数相同的项对应系数相加，若和不为“0”，则构成“和多项式”中的一项；
- 指数不同的项复制到“和多项式”中。

一元多项式相加的例题

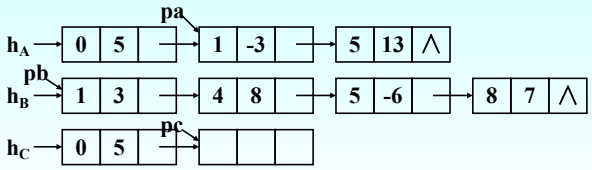
例：A(x)=5-3x+13x⁵，B(x)=3x+8x⁴-6x⁵+7x⁸，
求：C(x)=A(x)+B(x)。

解：a)pa->Exponent<pb->Exponent，则复制pa指向的结点到pc

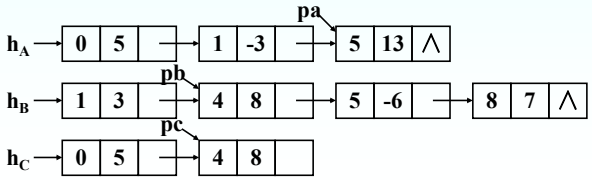


一元多项式相加的例题（续）

b)pa->Exponent=pb->Exponent，且pa->Coeff+pb->Coeff=0

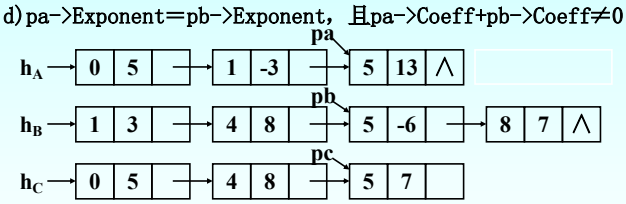


c)pa->Exponent>pb->Exponent，则复制pb

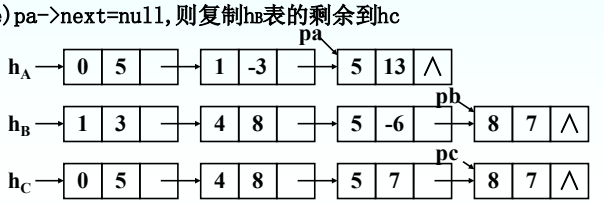


一元多项式相加的例题（续）

d)pa->Exponent=pb->Exponent，且pa->Coeff+pb->Coeff≠0



e)pa->next=null,则复制hB表的剩余到hc



思考题

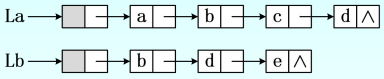
1.在一个具有n个结点的单链表，p 结点后插入一个新结点的时间复杂度是____；在给定值为x的结点后插入一个新结点的时间复杂度是_____。

A. O(1)　O(n)　　B. O(n)　O(1)
C. O(1)　O(1)　　D. O(n)　O(n)

2.若频繁地对线性表进行插入和删除操作，该线性表应该采用_____存储结构。

思考题

3. La和Lb是两个带头结点的单链表，如果La和Lb如下所示，画出Lc的结构图。




```
linklist * difference(linklist * La, linklist * Lb)
{   linklist *Lc, * pa, *pb, * s, * r;
    pa= La->next;
    Lc=(linklist * ) malloc (sizeof (linklist));
    r=Lc;
    while(pa! = NULL)
   {  pb=Lb-> next;
      while (pb! = NULL && pb-> data ! = pa-> data)
           pb= pb->next;
      if(pb = = NULL)
      {    s= (linklist * )malloe(sizeof(linklist));
           s-> data= pa->data;
           r->next=s;  r=s; }
       pa= pa->next;    }
    r->next = NULL;
    return Lc; }
```